

SQL Code

```
1  SELECT * FROM bm_stores LIMIT 10;
2  SELECT * FROM bm_skus LIMIT 10;
3  SELECT * FROM bm_customers LIMIT 10;
4  SELECT * FROM bm_inventory LIMIT 10;
5  SELECT * FROM bm_sales LIMIT 10;
6
7
8
9  -- Data Quality check and validation:
10 SELECT 'bm_stores' AS table_name, COUNT(*) AS row_count
11 FROM bm_stores
12
13 UNION ALL
14
15 SELECT 'bm_skus', COUNT(*)
16 FROM bm_skus
17
18 UNION ALL
19
20 SELECT 'bm_customers', COUNT(*)
21 FROM bm_customers
22
23 UNION ALL
24
25 SELECT 'bm_inventory', COUNT(*)
26 FROM bm_inventory
27
28 UNION ALL
29
30 SELECT 'bm_sales', COUNT(*)
31 FROM bm_sales;
32
33 -- Primary key validation:
34 SELECT
35 COUNT(*) AS total_rows,
```

```
36     COUNT(DISTINCT store_id) AS unique_store_ids
37   FROM bm_stores;
38
39   SELECT
40     COUNT(*) AS total_rows,
41     COUNT(DISTINCT sku_id) AS unique_sku_ids
42   FROM bm_skus;
43
44   SELECT
45     COUNT(*) AS total_rows,
46     COUNT(DISTINCT cust_id) AS unique_customer_ids
47   FROM bm_customers;
48
49   -- Inventory grain validation:
50   SELECT
51     COUNT(*) AS total_rows,
52     COUNT(DISTINCT (store_id, sku_id, snapshot_date)) AS unique_inventory_keys
53   FROM bm_inventory;
54
55   -- NULL validation:
56
57   -- Stores:
58   SELECT
59     COUNT(*) FILTER (WHERE store_id IS NULL) AS store_id_nulls,
60     COUNT(*) FILTER (WHERE store_name IS NULL) AS store_name_nulls,
61     COUNT(*) FILTER (WHERE city IS NULL) AS city_nulls,
62     COUNT(*) FILTER (WHERE store_type IS NULL) AS store_type_nulls
63   FROM bm_stores;
64
65   -- SKU:
66   SELECT
67     COUNT(*) FILTER (WHERE sku_id IS NULL) AS sku_id_nulls,
68     COUNT(*) FILTER (WHERE sku_name IS NULL) AS sku_name_nulls,
69     COUNT(*) FILTER (WHERE category IS NULL) AS category_nulls,
70     COUNT(*) FILTER (WHERE subcategory IS NULL) AS subcategory_nulls,
71     COUNT(*) FILTER (WHERE unit_price IS NULL) AS unit_price_nulls,
72     COUNT(*) FILTER (WHERE cost_price IS NULL) AS cost_price_nulls,
```

```
73     COUNT(*) FILTER (WHERE brand IS NULL) AS brand_nulls
74   FROM bm_skus;
75
76   -- Customers:
77   SELECT
78     COUNT(*) FILTER (WHERE cust_id IS NULL) AS cust_id_nulls,
79     COUNT(*) FILTER (WHERE age IS NULL) AS age_nulls,
80     COUNT(*) FILTER (WHERE gender IS NULL) AS gender_nulls,
81     COUNT(*) FILTER (WHERE city IS NULL) AS city_nulls,
82     COUNT(*) FILTER (WHERE loyalty_segment IS NULL) AS loyalty_segment_nulls,
83     COUNT(*) FILTER (WHERE preferred_channel IS NULL) AS preferred_channel_nulls,
84     COUNT(*) FILTER (WHERE registration_date IS NULL) AS registration_date_nulls
85   FROM bm_customers;
86
87
88   -- Sales:
89   SELECT
90     COUNT(*) AS total_rows,
91     COUNT(*) FILTER (WHERE date IS NULL) AS date_nulls,
92     COUNT(*) FILTER (WHERE store_id IS NULL) AS store_id_nulls,
93     COUNT(*) FILTER (WHERE sku_id IS NULL) AS sku_id_nulls,
94     COUNT(*) FILTER (WHERE customer_id IS NULL) AS customer_id_nulls,
95     COUNT(*) FILTER (WHERE quantity IS NULL) AS quantity_nulls,
96     COUNT(*) FILTER (WHERE unit_price IS NULL) AS unit_price_nulls,
97     COUNT(*) FILTER (WHERE total_value IS NULL) AS total_value_nulls,
98     COUNT(*) FILTER (WHERE channel IS NULL) AS channel_nulls,
99     COUNT(*) FILTER (WHERE discount_pct IS NULL) AS discount_pct_nulls
100  FROM bm_sales;
101
102
103  -- Foreign key Intergrity check:
104  SELECT COUNT(*) AS orphan_store_records
105  FROM bm_sales s
106  LEFT JOIN bm_stores st
107  ON s.store_id = st.store_id
108  WHERE st.store_id IS NULL;
109
```

```
110
111 SELECT COUNT(*) AS orphan_sku_records
112 FROM bm_sales s
113 LEFT JOIN bm_skus sk
114 ON s.sku_id = sk.sku_id
115 WHERE sk.sku_id IS NULL;
116
117 SELECT COUNT(*) AS orphan_customer_records
118 FROM bm_sales s
119 LEFT JOIN bm_customers c
120 ON s.customer_id = c.cust_id
121 WHERE s.customer_id IS NOT NULL
122 AND c.cust_id IS NULL;
123
124
125 -- Duplicates:
126 SELECT
127 COUNT(*) AS total_rows,
128 COUNT(DISTINCT (
129 date,
130 store_id,
131 sku_id,
132 customer_id,
133 quantity,
134 unit_price,
135 total_value,
136 channel,
137 discount_pct
138 )) AS unique_sales_records
139 FROM bm_sales;
140
141
142 SELECT
143 date,
144 store_id,
145 sku_id,
146 customer_id,
```

```
147     quantity,
148     unit_price,
149     total_value,
150     channel,
151     discount_pct,
152     COUNT(*) AS duplicate_count
153 FROM bm_sales
154 GROUP BY
155     date,
156     store_id,
157     sku_id,
158     customer_id,
159     quantity,
160     unit_price,
161     total_value,
162     channel,
163     discount_pct
164 HAVING COUNT(*) > 1
165 ORDER BY duplicate_count DESC
166 LIMIT 20;
167
168
169 -- Sales Value / Discount Validation:
170 SELECT
171     COUNT(*) AS total_rows,
172     COUNT(*) FILTER (
173     WHERE ABS(
174     (quantity * unit_price) - total_value
175     ) <= 0.01
176     ) AS matches_gross_value,
177     COUNT(*) FILTER (
178     WHERE ABS(
179     (quantity * unit_price * (1 - discount_pct / 100.0))
180     - total_value
181     ) <= 0.01
182     ) AS matches_discounted_value
183 FROM bm_sales;
```

```
184
185 -- Determine What unit_price Represents:
186 SELECT
187 COUNT(*) AS total_sales,
188 COUNT(*) FILTER (
189 WHERE s.unit_price = k.unit_price
190 ) AS exact_price_matches,
191 COUNT(*) FILTER (
192 WHERE s.unit_price <> k.unit_price
193 ) AS price_differences
194 FROM bm_sales s
195 JOIN bm_skus k
196 ON s.sku_id = k.sku_id;
197 -----
198 SELECT
199 discount_pct,
200 COUNT(*) AS sales_rows,
201 COUNT(*) FILTER (
202 WHERE s.unit_price = k.unit_price
203 ) AS price_matches,
204 COUNT(*) FILTER (
205 WHERE s.unit_price <> k.unit_price
206 ) AS price_differences
207 FROM bm_sales s
208 JOIN bm_skus k
209 ON s.sku_id = k.sku_id
210 GROUP BY discount_pct
211 ORDER BY discount_pct;
212
213
214 -- Business Rule:
215 SELECT
216 COUNT(*) AS total_rows,
217 COUNT(*) FILTER (WHERE quantity <= 0) AS invalid_quantity,
218 COUNT(*) FILTER (WHERE unit_price < 0) AS invalid_unit_price,
219 COUNT(*) FILTER (WHERE total_value < 0) AS invalid_total_value,
220 COUNT(*) FILTER (
```

```
221 WHERE discount_pct < 0 OR discount_pct > 100
222 ) AS invalid_discount
223 FROM bm_sales;
224
225
226 SELECT
227 COUNT(*) AS total_rows,
228 COUNT(*) FILTER (WHERE unit_price < 0) AS negative_unit_price,
229 COUNT(*) FILTER (WHERE cost_price < 0) AS negative_cost_price,
230 COUNT(*) FILTER (WHERE unit_price = 0) AS zero_unit_price,
231 COUNT(*) FILTER (WHERE cost_price = 0) AS zero_cost_price
232 FROM bm_skus;
233
234
235 SELECT
236 COUNT(*) AS total_rows,
237 COUNT(*) FILTER (WHERE stock_on_hand < 0) AS negative_stock,
238 COUNT(*) FILTER (WHERE reorder_point < 0) AS negative_reorder_point,
239 COUNT(*) FILTER (WHERE safety_stock < 0) AS negative_safety_stock
240 FROM bm_inventory;
241
242
243 -- Date Coverage:
244 SELECT
245 MIN(date) AS first_sales_date,
246 MAX(date) AS last_sales_date,
247 COUNT(DISTINCT date) AS sales_days
248 FROM bm_sales;
249
250 SELECT
251 MIN(snapshot_date) AS first_snapshot,
252 MAX(snapshot_date) AS last_snapshot,
253 COUNT(DISTINCT snapshot_date) AS snapshot_days
254 FROM bm_inventory;
255
256 SELECT
257 MIN(registration_date) AS first_registration,
```

```
258     MAX(registration_date) AS last_registration
259 FROM bm_customers;
260
261
262 -- Data Preparation and Transformation:
263 -----
264
265
266 CREATE OR REPLACE VIEW vw_dim_date AS
267 SELECT
268     d::date AS date,
269     EXTRACT(YEAR FROM d)::int AS year,
270     EXTRACT(QUARTER FROM d)::int AS quarter,
271     EXTRACT(MONTH FROM d)::int AS month_number,
272     TO_CHAR(d, 'Month') AS month_name,
273     EXTRACT(WEEK FROM d)::int AS week_number,
274     EXTRACT(DOW FROM d)::int AS day_of_week,
275     TO_CHAR(d, 'Day') AS day_name,
276     CASE
277     WHEN EXTRACT(DOW FROM d) IN (0, 6) THEN 'Weekend'
278     ELSE 'Weekday'
279     END AS day_type
280 FROM generate_series(
281     '2021-01-01'::date,
282     '2025-10-31'::date,
283     INTERVAL '1 day'
284 ) AS d;
285
286 -----
287
288 -- Create view Sales_enriched:
289
290 CREATE OR REPLACE VIEW vw_sales_enriched AS
291 SELECT
292     s.date,
293     s.store_id,
294     st.store_name,
```

```
295     st.city AS store_city,
296     st.store_type,
297
298     s.skus_id,
299     sk.skus_name,
300     sk.category,
301     sk.subcategory,
302     sk.brand,
303
304     s.customer_id,
305     CASE
306     WHEN s.customer_id IS NULL THEN 'Unidentified Customer'
307     ELSE 'Identified Customer'
308     END AS customer_status,
309
310     c.loyalty_segment,
311     c.preferred_channel,
312
313     s.channel,
314     s.quantity,
315     sk.unit_price AS list_price,
316     s.unit_price AS selling_price,
317     s.discount_pct,
318     s.total_value AS net_sales,
319
320     -- Value before transaction discount
321     s.quantity * sk.unit_price AS gross_list_value,
322
323     -- Value given up through discounting
324     (s.quantity * sk.unit_price) - s.total_value AS discount_amount,
325
326     -- Product cost
327     s.quantity * sk.cost_price AS product_cost,
328
329     -- Gross profit
330     s.total_value - (s.quantity * sk.cost_price) AS gross_profit,
331
```

```
332  -- Gross margin %
333  CASE
334  WHEN s.total_value <> 0 THEN
335  (
336  (s.total_value - (s.quantity * sk.cost_price))
337  / s.total_value
338  ) * 100
339  ELSE NULL
340  END AS gross_margin_pct
341  FROM bm_sales s
342  LEFT JOIN bm_stores st
343  ON s.store_id = st.store_id
344  LEFT JOIN bm_skus sk
345  ON s.sku_id = sk.sku_id
346  LEFT JOIN bm_customers c
347  ON s.customer_id = c.cust_id;
348
349  /*
350  Original sales
351  +
352  Store attributes
353  +
354  SKU attributes
355  +
356  Customer attributes
357  +
358  Profitability metrics
359  ↓
360  Analytical sales layer
361  */
362  -----
363
364  -- Create view Inventory Risk:
365  CREATE OR REPLACE VIEW vw_inventory_risk AS
366  SELECT
367  i.snapshot_date,
368  i.store_id,
```

```
369     st.store_name,
370     st.city AS store_city,
371     st.store_type,
372
373     i.sku_id,
374     sk.sku_name,
375     sk.category,
376     sk.subcategory,
377     sk.brand,
378
379     i.stock_on_hand,
380     i.reorder_point,
381     i.safety_stock,
382     i.last_restock_date,
383
384     CASE
385     WHEN i.stock_on_hand < i.safety_stock
386     THEN 'Below Safety Stock'
387     WHEN i.stock_on_hand < i.reorder_point
388     THEN 'Below Reorder Point'
389     ELSE 'Normal'
390     END AS inventory_status
391 FROM bm_inventory i
392 LEFT JOIN bm_stores st
393 ON i.store_id = st.store_id
394 LEFT JOIN bm_skus sk
395 ON i.sku_id = sk.sku_id;
396
397 -- stock < safety_stock --
398 -----
399
400 -- KPI and metrics baseline:
401 SELECT
402 COUNT(*) AS sales_records,
403 SUM(quantity) AS units_sold,
404 SUM(net_sales) AS net_sales,
405 SUM(gross_list_value) AS gross_list_value,
```

```
406     SUM(discount_amount) AS discount_amount,
407     SUM(product_cost) AS product_cost,
408     SUM(gross_profit) AS gross_profit,
409
410     CASE
411     WHEN SUM(net_sales) <> 0
412     THEN SUM(gross_profit) / SUM(net_sales) * 100
413     ELSE 0
414     END AS gross_margin_pct,
415
416     CASE
417     WHEN SUM(gross_list_value) <> 0
418     THEN SUM(discount_amount) / SUM(gross_list_value) * 100
419     ELSE 0
420     END AS discount_rate,
421
422     CASE
423     WHEN SUM(quantity) <> 0
424     THEN SUM(net_sales) / SUM(quantity)
425     ELSE 0
426     END AS avg_selling_price
427 FROM vw_sales_enriched;
428
429 -- Customer baseline:
430 SELECT
431     COUNT(*) AS sales_records,
432     COUNT(*) FILTER (
433     WHERE customer_id IS NOT NULL
434     ) AS identified_sales_records,
435     COUNT(*) FILTER (
436     WHERE customer_id IS NULL
437     ) AS unidentified_sales_records,
438
439     COUNT(DISTINCT customer_id) AS identified_customers,
440
441     ROUND(
442     COUNT(*) FILTER (
```

```
443 WHERE customer_id IS NOT NULL
444 ) * 100.0 / COUNT(*),
445 2
446 ) AS customer_identification_rate_pct
447 FROM bm_sales;
448
449
450
451
452 -- Inventory Baseline:
453 SELECT
454 COUNT(*) AS inventory_records,
455 SUM(stock_on_hand) AS total_stock_on_hand,
456
457 COUNT(*) FILTER (
458 WHERE stock_on_hand < reorder_point
459 ) AS below_reorder_point,
460
461 COUNT(*) FILTER (
462 WHERE stock_on_hand < safety_stock
463 ) AS below_safety_stock,
464
465 ROUND(
466 COUNT(*) FILTER (
467 WHERE stock_on_hand < reorder_point
468 ) * 100.0 / COUNT(*),
469 2
470 ) AS reorder_risk_pct,
471
472 ROUND(
473 COUNT(*) FILTER (
474 WHERE stock_on_hand < safety_stock
475 ) * 100.0 / COUNT(*),
476 2
477 ) AS safety_stock_risk_pct
478 FROM bm_inventory;
```